GPX Programming Language

Guided Programming eXperience

Official Language Specification, Architecture & Reference Manual

---

Version 0.9.1

Lead Architect:
Nishit (@nishit0072e)

Release Date:
October 2026

> *"Most people look at compilers and see cold, impenetrable black boxes. We looked at them and asked an audacious question: What if compiling code wasn't a chore, but an exhilarating journey of discovery? What if every developer could peek behind the curtain of modern silicon and feel the raw pulse of the machine?"*

Copyright © 2026 Nishit. All Rights Reserved. Proprietary & Confidential.

Contents

1 Introduction & Design Philosophy **2**
1.1 Core Tenets 2

2 Lexical Structure & Grammar **3**
2.1 Keywords 3
2.2 Type System 3

3 Object-Oriented Programming (OOP) in GPX **4**

4 Data Structures & Algorithms (DSA) Suite **5**

5 Interactive REPL, Audio Synthesizer & Debugger **6**
5.1 Interactive REPL (`gpx -repl`) 6
5.2 8-Bit Audio Synthesis (`<sound.gpx>`) 6
5.3 Real-Time Memory & Instruction Inspector (`gpx -step` / `gpx-dbg`) 6

6 Interactive Story & Adventure Tutorial **7**

7 CLI Command Reference **8**

Chapter 1

Introduction & Design Philosophy

The **Guided Programming eXperience (GPX)** is a statically-typed, target-agnostic systems programming language designed for developers, systems engineers, and students who crave bare-metal speed without cognitive obscurity.

1.1 Core Tenets

- **Total Pipeline Introspectability:** Every transformation in GPX—from token streams (`-tokens`) and Abstract Syntax Trees (`-ast`) to Three-Address Code (`-ir`), optimization passes (`-opt`), and machine code—can be inspected with a single flag.
- **Zero-Overhead Runtime:** GPX executes without garbage collection pauses, heavy runtimes, or hidden virtualization penalties.
- **Dual Audience Design:** GPX seamlessly bridges the gap between educational accessibility (friendly loop constructs, turtle graphics, interactive sound synthesis) and rigorous systems programming (raw pointers, unions, direct memory arenas, and RISC-V assembly generation).
- **Pluggable Multi-Backend Compilation:** Emits high-performance ISO C99 (compiled with GCC) and bare-metal RISC-V RV32I assembly.

Chapter 2

Lexical Structure & Grammar

2.1 Keywords

The following tokens are reserved keywords in GPX:

`let`	`const`	`fn`	`return`	`if`
`else`	`while`	`for`	`repeat`	`when`
`break`	`continue`	`struct`	`union`	`enum`
`as`	`sizeof`	`null`	`true`	`false`

2.2 Type System

GPX is statically typed with explicit annotations and local type inference:

- **Primitives:** `int` (32-bit), `long` (64-bit), `short` (16-bit), `byte` (8-bit), `char` (8-bit ASCII), `float` (32-bit IEEE 754), `double` (64-bit IEEE 754), `bool`, `void`, `string`.
- **Pointers:** Pointers are denoted with an asterisk prefix (e.g., `*int`, `*void`, `*Player`). The address-of operator is `&`, and dereferencing is performed via `*` or arrow access `->`.
- **Fixed-Size Arrays:** Sized arrays use the syntax `[T; N]`, with zero-cost indexed access `arr[i]`.
- **Composite Types:** User-defined records via `struct`, `union`, and scoped `enum`.

Chapter 3

Object-Oriented Programming (OOP) in GPX

GPX adopts a zero-overhead receiver method syntax modeled after modern systems languages:

```
struct Vector2D {
    x: double,
    y: double,
}

// Method with pointer receiver
fn (v: *Vector2D) translate(dx: double, dy: double) {
    v->x = v->x + dx;
    v->y = v->y + dy;
}

fn (v: *Vector2D) length_squared() -> double {
    return v->x * v->x + v->y * v->y;
}

fn main() -> int {
    let pos = Vector2D { x: 10.0, y: 20.0 };
    pos.translate(5.0, -2.0);
    print(pos.length_squared());
    return 0;
}
```

Listing 3.1: Receiver Method and Struct Definition

Chapter 4

Data Structures & Algorithms (DSA) Suite

GPX includes a high-performance, modular standard library for core algorithms and collections:

`<list.gpx>`: Contiguous growable dynamic array with geometric capacity scaling (2×).

`<linkedlist.gpx>`: Singly and doubly linked node structures for constant-time head/tail insertion.

`<stack.gpx>` & `<queue.gpx>`: LIFO stack and FIFO circular ring buffer.

`<hashmap.gpx>`: Key-value associative map with open-addressing and FNV-1a hashing.

`<tree.gpx>`: Self-balancing and binary search trees with recursive traversals.

`<graph.gpx>`: Adjacency-list graphs with Breadth-First (BFS) and Depth-First (DFS) search.

`<algorithms.gpx>`: Quicksort, Mergesort, Insertion sort, and Binary search.

`<pool.gpx>`: O(1) fixed-size block memory pool allocator for deterministic embedded targets.

Chapter 5

Interactive REPL, Audio Synthesizer & Debugger

5.1 Interactive REPL (`gpx -repl`)

The GPX shell provides immediate feedback with type reflection (`type(x)`), state persistence across inputs, and meta commands (`.vars`, `.funcs`, `.mem`, `.ir`, `.ast`, `.load`).

5.2 8-Bit Audio Synthesis (`<sound.gpx>`)

Direct hardware audio synthesis without external dependencies:

- Real-time tones: `sound_beep(freq, duration)`
- Note notation: `sound_play_note("C4", 250)`
- Retro game SFX: `sound_sfx_coin()`, `sound_sfx_laser()`, `sound_sfx_jump()`
- Multi-wave synthesis: Square, Sine, Triangle, Sawtooth, and Noise with direct WAV export.

5.3 Real-Time Memory & Instruction Inspector (`gpx -step` / `gpx-dbg`)

An interactive debugger providing:

- Time-travel step rewinding (`b`)
- Live Three-Address Code (TAC) instruction cursor
- Real-time heap block visualization (allocation id, size, freed status, raw words)
- Full Debug Adapter Protocol (DAP) compliance for VS Code and Antigravity IDE.

Chapter 6

Interactive Story & Adventure Tutorial

Launched via `gpx -tutorial`, the adventure mode (*"The Chronicles of the Motherboard"*) features 7 gamified quest chapters:

1. **The Awakening:** Core generator power calculation and integer arithmetic.
2. **The Security Gate:** Cryptographic cipher door with boolean branching.
3. **The Power Turbine:** Cycling hyperdrive turbine coils via loop accumulation.
4. **The Star Beacon:** Projecting 2D turtle vector diagrams into space.
5. **Signal of Hope:** Transmitting 8-bit audio homing frequencies.
6. **The Autonomous Drone:** Constructing robots with OOP struct receiver methods.
7. **The Data Core:** Mainframe stabilization using dynamic arrays (`<list.gpx>`).

Chapter 7

CLI Command Reference

Category	Flag / Command	Description
Execution	`-run`	Direct Virtual Machine execution without intermediate files
	`-repl`	Launches the interactive shell
	`-tutorial`	Launches the gamified Story / Adventure Mode
Debugging	`-step` / `-debugger`	Launches the interactive Memory Inspector
	`-dap`	Launches the headless Debug Adapter Protocol server
Inspection	`-tokens`	Emits token stream
	`-ast`	Emits Abstract Syntax Tree hierarchy
	`-ir`	Emits Three-Address Code (TAC) IR
	`-opt`	Emits optimized TAC IR
	`-check`	Semantic analysis and type checking
Codegen	`-o <file.exe>`	Compiles to standalone native binary via GCC
	`-emit-c`	Transpiles to ISO C99
	`-target riscv`	Emits bare-metal RISC-V RV32I assembly

Table 7.1: GPX Compiler Command Line Interface